

Orchfile Grammar Reference

Orchfile Specification

This document contains the formal grammar for Orchfile in W3C EBNF notation, followed by railroad (syntax) diagrams for the key production rules. The canonical source is `grammar.ebnf` in the repository root.

Contents

1	EBNF Grammar	2
1.1	Document Structure	2
1.2	Top-level Declarations	2
1.3	Directives	2
1.3.1	Overlay Composition	2
1.3.2	Execution Mode (mutually exclusive per service)	2
1.3.3	Container-only Directives	2
1.3.4	Host-only Directives	2
1.3.5	Common Directives	3
1.4	Terminals	3
2	Railroad Diagrams	5
3	Constraints	7

1 EBNF Grammar

1.1 Document Structure

```
orchfile      ::= ( line newline ) *
line          ::= blank | comment | arg | service_decl | directive
blank         ::= ws *
comment       ::= ws * '#' any *
```

1.2 Top-level Declarations

```
arg           ::= 'ARG' ws identifier '=' value
service_decl ::= 'SERVICE' ws service_name
```

1.3 Directives

```
directive     ::= mode_directive
               | container_directive
               | host_directive
               | common_directive
               | clear_directive
```

1.3.1 Overlay Composition

```
clear_directive
    ::= 'CLEAR' ws clearable_target

clearable_target
    ::= 'ENV' | 'ENV_FILE' | 'PUBLISH' | 'VOLUME'
       | 'REQUIRES' | 'AFTER'
```

1.3.2 Execution Mode (mutually exclusive per service)

```
mode_directive
    ::= 'FROM' ws image_ref
       | 'RUN' ws command_string
```

1.3.3 Container-only Directives

```
container_directive
    ::= 'ENTRYPOINT' ws command_string
       | 'CMD' ws command_string
       | 'PUBLISH' ws port ':' port
       | 'VOLUME' ws volume_source ':' path
```

1.3.4 Host-only Directives

```
host_directive
    ::= 'USER' ws identifier
       | 'STOP' ws command_string
       | 'RELOAD' ws command_string
```

1.3.5 Common Directives

```
common_directive
    ::= env_directive
       | dependency_directive
       | health_directive
       | lifecycle_directive
       | restart_directive
       | timeout_directive
       | resource_directive
       | logging_directive

env_directive
    ::= 'WORKDIR' ws path
       | 'ENV' ws env_key '=' value
       | 'ENV_FILE' ws path

dependency_directive
    ::= 'REQUIRES' ws service_name ( ws service_name ) *
       | 'AFTER' ws service_name ( ws service_name ) *

health_directive
    ::= 'HEALTHCHECK' ws ( url | command_string )
       | 'READINESS_TIMEOUT' ws duration

lifecycle_directive
    ::= 'ONESHOT' ws bool
       | 'DISABLED' ws bool
       | 'RECREATE' ws recreate_policy

restart_directive
    ::= 'RESTART' ws restart_policy
       | 'RESTART_DELAY' ws duration
       | 'START_LIMIT_BURST' ws integer
       | 'START_LIMIT_INTERVAL' ws duration

timeout_directive
    ::= 'TIMEOUT_START' ws duration
       | 'TIMEOUT_STOP' ws duration

resource_directive
    ::= 'MEMORY' ws memory_size
       | 'CPUS' ws number
       | 'CPU_QUOTA' ws percentage
       | 'LIMIT_NOFILE' ws integer
       | 'LIMIT_NPROC' ws integer
       | 'TASKS_MAX' ws integer
       | 'IO_WEIGHT' ws integer /* 10-1000 */

logging_directive
    ::= 'STDOUT' ws path
       | 'STDERR' ws path
```

1.4 Terminals

```
service_name    ::= letter ( letter | digit | '-' ) * /* max 63 chars */
identifier      ::= letter ( letter | digit | '-' ) *
```

```

env_key          ::= ( letter / '_' ) ( letter / digit / '_' ) *

image_ref        ::= ( any - newline ) +
command_string   ::= ( any - newline ) +
path             ::= ( any - newline ) +
value            ::= ( any - newline ) *
url              ::= ( 'http://' / 'https://' ) ( any - newline ) +

var_ref          ::= '${ identifier }'

bool             ::= 'true' / 'false'
restart_policy   ::= 'no' / 'always' / 'on-failure'
recreate_policy  ::= 'always' / 'never'

duration         ::= integer ( 's' / 'm' )
memory_size      ::= integer ( 'K' / 'M' / 'G' )
percentage       ::= integer '%'
number           ::= integer ( '.' digit + ) ?
integer          ::= digit +
port             ::= digit + /* 1-65535 */

letter           ::= [a-z]
digit            ::= [0-9]
ws               ::= [ \t ] +
newline          ::= \n
any              ::= [U+0020-U+FFFF]

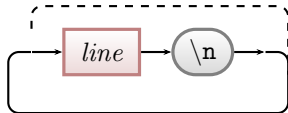
```

Note: `var_ref` (`${name}`) may appear within any `value`, `path`, `image_ref`, or `command_string`. Variable expansion is performed at parse time using resolved ARG values. Unresolved references to built-in variables (e.g. `${ORCH_PROJECT}`) are preserved for runtime resolution.

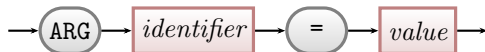
2 Railroad Diagrams

The following diagrams visualize the key production rules. Rounded boxes are *terminals* (literal text), rectangular boxes are *non-terminals* (references to other rules).

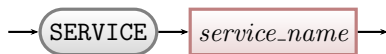
orchfile



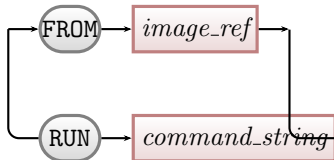
arg



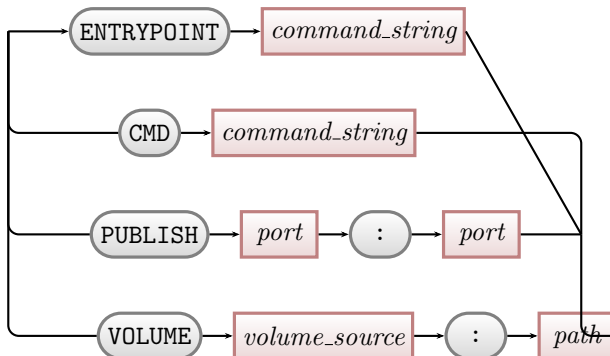
service_decl



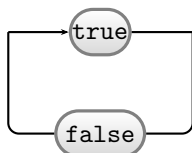
mode_directive



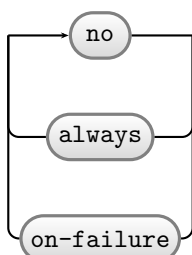
container_directive



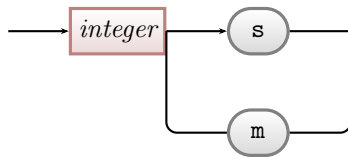
bool



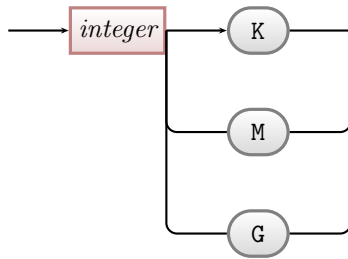
restart_policy



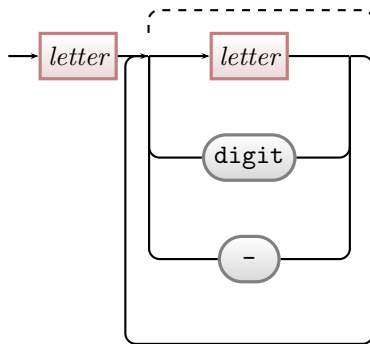
duration



memory_size



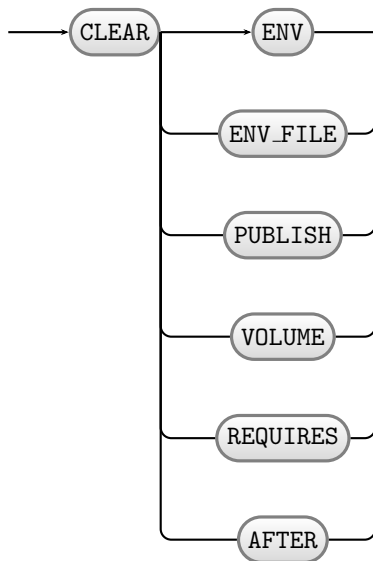
service_name (max 63 characters)



var_ref



clear_directive



3 Constraints

The following constraints are enforced by the parser and cannot be expressed in the EBNF grammar alone:

- C1: FROM XOR RUN** A service must specify exactly one of FROM (container mode) or RUN (host mode). Both or neither is an error.
- C2: Container-only directives** ENTRYPOINT, CMD, PUBLISH, VOLUME are only valid with FROM.
- C3: Host-only directives** USER, STOP, RELOAD are only valid with RUN.
- C4: Dependency acyclicity** REQUIRES and AFTER edges must form a directed acyclic graph (DAG). Cycles are a parse error.